8. 저장소 시스템

8.1 저장소 아키텍처 개요

Narwhal의 저장소 시스템은 인증서, 배치, 컨센서스 상태 등을 영구적으로 보관하는 핵심 구성 요소입니다.

8.1.1 저장소 계층 구조

```
Application Layer | Certificate Store
                  | Batch Store
                  | Consensus Store
──────────────────┼──────────────────
Abstraction Layer | Store Traits
                  | Typed Store
                  | Cache Layer
──────────────────┼──────────────────
Storage Engine    | RocksDB
                  | Memory Store
                  | File System
```

8.1.2 주요 저장소 타입

- **CertificateStore**: 인증서 영구 저장

- **BatchStore**: 트랜잭션 배치 저장
- **ConsensusStore**: 컨센서스 상태 저장
- **ConfigStore**: 설정 정보 저장

8.2 저장소 추상화 계층

8.2.1 Store Trait 정의

```rust
use serde::{Deserialize, Serialize};
use std::collections::HashMap;

pub trait Store<K, V> {
type Error;
fn read(&self, key: &K) -> Result<Option<V>, Self::Error>;
fn write(&self, key: &K, value: &V) -> Result<(), Self::Error>;
fn delete(&self, key: &K) -> Result<(), Self::Error>;
fn iter(&self) -> Result<Box<dyn Iterator<Item = (K, V)>>, Self::Error>;
fn batch_write(&self, batch: &[(K, V)]) -> Result<(), Self::Error>;
}

pub trait TypedStore<V> {
type Key;
type Error;
fn read(&self, key: &Self::Key) -> Result<Option<V>, Self::Error>;
```

```
fn write(&self, key: &Self::Key, value: &V) -> Result<(), Self::Error>;
fn delete(&self, key: &Self::Key) -> Result<(), Self::Error>;
}
```

8.2.2 RocksDB 구현

```
use rocksdb::{DB, Options, WriteBatch};
use bincode;

pub struct RocksDBStore {
db: DB,
column_family: String,
}

impl RocksDBStore {
pub fn new(path: &str, cf_name: &str) -> Result<Self, StoreError> {
let mut opts = Options::default();
opts.create_if_missing(true);
opts.create_missing_column_families(true);
let db = DB::open_cf(&opts, path, &[cf_name])?;
Ok(RocksDBStore {
db,
column_family: cf_name.to_string(),
```

```
})
}
}


impl<K, V> Store<K, V> for RocksDBStore
where
K: Serialize + for<'de> Deserialize<'de>,
V: Serialize + for<'de> Deserialize<'de>,
{
type Error = StoreError;
fn read(&self, key: &K) -> Result<Option<V>, Self::Error> {
let cf = self.db.cf_handle(&self.column_family)
.ok_or(StoreError::ColumnFamilyNotFound)?;
let key_bytes = bincode::serialize(key)?;
match self.db.get_cf(cf, &key_bytes)? {
Some(value_bytes) => {
let value = bincode::deserialize(&value_bytes)?;
Ok(Some(value))
}
None => Ok(None),
}
}
fn write(&self, key: &K, value: &V) -> Result<(), Self::Error> {
```

```
let cf = self.db.cf_handle(&self.column_family)
.ok_or(StoreError::ColumnFamilyNotFound)?;
let key_bytes = bincode::serialize(key)?;
let value_bytes = bincode::serialize(value)?;
self.db.put_cf(cf, &key_bytes, &value_bytes)?;
Ok(())
}
fn batch_write(&self, batch: &[(K, V)]) -> Result<(), Self::Error> {
let cf = self.db.cf_handle(&self.column_family)
.ok_or(StoreError::ColumnFamilyNotFound)?;
let mut write_batch = WriteBatch::default();
for (key, value) in batch {
let key_bytes = bincode::serialize(key)?;
let value_bytes = bincode::serialize(value)?;
write_batch.put_cf(cf, &key_bytes, &value_bytes);
}
self.db.write(write_batch)?;
Ok(())
}
}
```

8.3 인증서 저장소 (CertificateStore)

8.3.1 인증서 저장 구조

```
pub struct CertificateStore {
store: RocksDBStore,
cache: LruCache<CertificateDigest, Certificate>,
metrics: Arc<StorageMetrics>,
}

impl CertificateStore {
pub fn new(path: &str, cache_size: usize) -> Result<Self, StoreError> {
let store = RocksDBStore::new(path, "certificates")?;
let cache = LruCache::new(cache_size);
let metrics = Arc::new(StorageMetrics::new());
Ok(CertificateStore {
store,
cache,
metrics,
})
}
pub fn write_certificate(&mut self, certificate: Certificate) -> Result<(), S
let digest = certificate.digest();
// 캐시에 저장
self.cache.put(digest, certificate.clone());
```

```
// 영구 저장소에 저장
self.store.write(&digest, &certificate)?;
// 메트릭 업데이트
self.metrics.certificates_written.inc();
Ok(())
}
pub fn read_certificate(&mut self, digest: &CertificateDigest) -> Result<Opti
// 캐시에서 먼저 확인
if let Some(certificate) = self.cache.get(digest) {
self.metrics.cache_hits.inc();
return Ok(Some(certificate.clone()));
}
// 영구 저장소에서 읽기
match self.store.read(digest)? {
Some(certificate) => {
// 캐시에 저장
self.cache.put(*digest, certificate.clone());
self.metrics.cache_misses.inc();
Ok(Some(certificate))
}
None => Ok(None),
}
}
```

```
pub fn certificates_by_round(&self, round: Round) -> Result<Vec<Certificate>,
let mut certificates = Vec::new();
// 라운드별 인덱스를 사용하여 효율적 조회
for result in self.store.iter()? {
let (_, certificate) = result;
if certificate.round() == round {
certificates.push(certificate);
}
}
Ok(certificates)
}
pub fn after_round(&self, round: Round) -> Result<Vec<Certificate>, StoreError
let mut certificates = Vec::new();
for result in self.store.iter()? {
let (_, certificate) = result;
if certificate.round() > round {
certificates.push(certificate);
}
}
Ok(certificates)
}
}
```

8.3.2 인덱싱 시스템

```
pub struct CertificateIndex {
// 라운드별 인덱스
round_index: HashMap<Round, Vec<CertificateDigest>>,
// 작성자별 인덱스
author_index: HashMap<PublicKey, Vec<CertificateDigest>>,
// 부모-자식 관계 인덱스
parent_child_index: HashMap<CertificateDigest, Vec<CertificateDigest>>,
}

impl CertificateIndex {
pub fn add_certificate(&mut self, certificate: &Certificate) {
let digest = certificate.digest();
let round = certificate.round();
let author = certificate.origin();
// 라운드 인덱스 업데이트
self.round_index.entry(round)
.or_insert_with(Vec::new)
.push(digest);
// 작성자 인덱스 업데이트
self.author_index.entry(author)
.or_insert_with(Vec::new)
```

```
.push(digest);
// 부모-자식 관계 업데이트
for parent_digest in &certificate.header.parents {
self.parent_child_index.entry(*parent_digest)
.or_insert_with(Vec::new)
.push(digest);
}
}
pub fn certificates_by_round(&self, round: Round) -> Option<&Vec<CertificateDi
self.round_index.get(&round)
}
pub fn certificates_by_author(&self, author: &PublicKey) -> Option<&Vec<Certif
self.author_index.get(author)
}
}
```

8.4 배치 저장소 (BatchStore)

8.4.1 배치 저장 구조

```
pub struct BatchStore {
store: RocksDBStore,
transaction_index: HashMap<TransactionDigest, BatchDigest>,
```

```
gc_policy: GarbageCollectionPolicy,
metrics: Arc<StorageMetrics>,
}


impl BatchStore {
pub fn write_batch(&mut self, batch: Batch) -> Result<BatchDigest, StoreError
let digest = batch.digest();
// 배치 저장
self.store.write(&digest, &batch)?;
// 트랜잭션 인덱스 업데이트
for transaction in &batch.transactions {
let tx_digest = transaction.digest();
self.transaction_index.insert(tx_digest, digest);
}
self.metrics.batches_written.inc();
Ok(digest)
}
pub fn read_batch(&self, digest: &BatchDigest) -> Result<Option<Batch>, StoreE
match self.store.read(digest)? {
Some(batch) => {
self.metrics.batches_read.inc();
Ok(Some(batch))
}
```

```
None => Ok(None),
}
}
pub fn find_batch_by_transaction(&self, tx_digest: &TransactionDigest) -> Opti
self.transaction_index.get(tx_digest).copied()
}
pub fn cleanup_old_batches(&mut self, cutoff_time: SystemTime) -> Result<usiz
let mut removed_count = 0;
let mut to_remove = Vec::new();
// 오래된 배치 식별
for result in self.store.iter()? {
let (digest, batch) = result;
if batch.timestamp < cutoff_time {
to_remove.push(digest);
}
}
// 배치 삭제
for digest in to_remove {
self.store.delete(&digest)?;
removed_count += 1;
}
self.metrics.batches_garbage_collected.add(removed_count as u64);
Ok(removed_count)
```

```
}

}
```

8.4.2 트랜잭션 인덱싱

```
pub struct TransactionIndex {

// 트랜잭션 → 배치 매핑

tx_to_batch: HashMap<TransactionDigest, BatchDigest>,

// 발신자별 트랜잭션 인덱스

sender_index: HashMap<PublicKey, Vec<TransactionDigest>>,

// 시간별 트랜잭션 인덱스

time_index: BTreeMap<SystemTime, Vec<TransactionDigest>>,

}


impl TransactionIndex {

pub fn add_batch(&mut self, batch: &Batch) {

let batch_digest = batch.digest();

for transaction in &batch.transactions {

let tx_digest = transaction.digest();

// 트랜잭션 → 배치 매핑

self.tx_to_batch.insert(tx_digest, batch_digest);

// 발신자별 인덱스

self.sender_index.entry(transaction.sender())
```

```
.or_insert_with(Vec::new)
.push(tx_digest);
// 시간별 인덱스
self.time_index.entry(batch.timestamp)
.or_insert_with(Vec::new)
.push(tx_digest);
}
}
pub fn transactions_by_sender(&self, sender: &PublicKey) -> Option<&Vec<Transa
self.sender_index.get(sender)
}
pub fn transactions_after_time(&self, time: SystemTime) -> Vec<TransactionDige
self.time_index.range(time..)
.flat_map(|(_, txs)| txs.iter())
.copied()
.collect()
}
}
```

8.5 컨센서스 저장소 (ConsensusStore)

8.5.1 컨센서스 상태 저장

```rust
pub struct ConsensusStore {
store: RocksDBStore,
state_cache: Arc<RwLock<ConsensusStateCache>>,
}

struct ConsensusStateCache {
last_consensus_index: SequenceNumber,
last_committed: HashMap<PublicKey, Round>,
committed_certificates: LruCache<CertificateDigest, Certificate>,
}

impl ConsensusStore {
pub fn write_consensus_state(
&self,
last_committed: &HashMap<PublicKey, Round>,
consensus_index: &SequenceNumber,
certificate_digest: &CertificateDigest,
) -> Result<(), StoreError> {
// 상태를 배치로 저장
let batch = vec![
(StateKey::LastCommitted, StateValue::LastCommitted(last_committed.clone())),
(StateKey::ConsensusIndex, StateValue::ConsensusIndex(*consensus_index)),
```

```
(StateKey::LastCertificate, StateValue::CertificateDigest(*certificate_digest)),
];
self.store.batch_write(&batch)?;
// 캐시 업데이트
{
let mut cache = self.state_cache.write().unwrap();
cache.last_consensus_index = *consensus_index;
cache.last_committed = last_committed.clone();
}
Ok(())
}
pub fn read_last_consensus_index(&self) -> Result<SequenceNumber, StoreError>
// 캐시에서 먼저 확인
{
let cache = self.state_cache.read().unwrap();
if cache.last_consensus_index > 0 {
return Ok(cache.last_consensus_index);
}
}
// 저장소에서 읽기
match self.store.read(&StateKey::ConsensusIndex)? {
Some(StateValue::ConsensusIndex(index)) => {
// 캐시 업데이트
```

```
{
let mut cache = self.state_cache.write().unwrap();
cache.last_consensus_index = index;
}
Ok(index)
}
_ => Ok(0), // 초기값
}
}
pub fn read_last_committed(&self) -> HashMap<PublicKey, Round> {
// 캐시에서 먼저 확인
{
let cache = self.state_cache.read().unwrap();
if !cache.last_committed.is_empty() {
return cache.last_committed.clone();
}
}
// 저장소에서 읽기
match self.store.read(&StateKey::LastCommitted) {
Ok(Some(StateValue::LastCommitted(committed))) => {
// 캐시 업데이트
{
let mut cache = self.state_cache.write().unwrap();
```

```
cache.last_committed = committed.clone();
}
committed
}
_ => HashMap::new(), // 초기값
}
}
pub fn clear(&self) -> Result<(), StoreError> {
// 모든 상태 키 삭제
let keys_to_delete = vec![
StateKey::LastCommitted,
StateKey::ConsensusIndex,
StateKey::LastCertificate,
];
for key in keys_to_delete {
self.store.delete(&key)?;
}
// 캐시 초기화
{
let mut cache = self.state_cache.write().unwrap();
cache.last_consensus_index = 0;
cache.last_committed.clear();
cache.committed_certificates.clear();
```

```
}
Ok(())
}
}
```

8.5.2 상태 키-값 정의

```
#[derive(Serialize, Deserialize, Clone, Debug, PartialEq, Eq, Hash)]
pub  enum StateKey {
LastCommitted,
ConsensusIndex,
LastCertificate,
EpochInfo,
CommitteeInfo,
}

#[derive(Serialize, Deserialize, Clone, Debug)]
pub  enum StateValue {
LastCommitted(HashMap<PublicKey, Round>),
ConsensusIndex(SequenceNumber),
CertificateDigest(CertificateDigest),
EpochInfo(Epoch),
CommitteeInfo(Committee),
```

```
}
```

8.6 성능 최적화

8.6.1 캐싱 전략

```
pub struct MultiLevelCache {
l1_cache: LruCache<CertificateDigest, Certificate>, // 메모리 캐시
l2_cache: Option<RedisCache>, // 분산 캐시
cache_policy: CachePolicy,
}

impl MultiLevelCache {
pub fn get(&mut self, key: &CertificateDigest) -> Option<Certificate> {
// L1 캐시 확인
if let Some(value) = self.l1_cache.get(key) {
return Some(value.clone());
}
// L2 캐시 확인
if let Some(ref l2) = self.l2_cache {
if let Ok(Some(value)) = l2.get(key) {
// L1 캐시에 저장
self.l1_cache.put(*key, value.clone());
```

```
return Some(value);
}
}
None
}
pub fn put(&mut self, key: CertificateDigest, value: Certificate) {
// L1 캐시에 저장
self.l1_cache.put(key, value.clone());
// L2 캐시에 저장 (비동기)
if let Some(ref l2) = self.l2_cache {
let _ = l2.put_async(key, value);
}
}
}
```

8.6.2 압축과 직렬화 최적화

```
pub struct OptimizedSerialization {
compression_enabled: bool,
compression_threshold: usize,
serialization_format: SerializationFormat,
}
```

```
impl OptimizedSerialization {
pub fn serialize<T: Serialize>(&self, value: &T) -> Result<Vec<u8>, Serializat
let data = match self.serialization_format {
SerializationFormat::Bincode => bincode::serialize(value)?,
SerializationFormat::MessagePack => rmp_serde::to_vec(value)?,
SerializationFormat::Cbor => serde_cbor::to_vec(value)?,
};
if self.compression_enabled && data.len() > self.compression_threshold {
Ok(self.compress(&data)?)
} else {
Ok(data)
}
}
fn compress(&self, data: &[u8]) -> Result<Vec<u8>, CompressionError> {
use flate2::write::GzEncoder;
use flate2::Compression;
use std::io::Write;
let mut encoder = GzEncoder::new(Vec::new(), Compression::default());
encoder.write_all(data)?;
Ok(encoder.finish()?)
}
}
```

8.7 백업과 복구

8.7.1 스냅샷 생성

```
pub struct SnapshotManager {
store: Arc<CertificateStore>,
snapshot_interval: Duration,
max_snapshots: usize,
}

impl SnapshotManager {
pub async fn create_snapshot(&self, round: Round) -> Result<SnapshotId, Snaps
let snapshot_id = SnapshotId::new(round, SystemTime::now());
let snapshot_path = self.get_snapshot_path(&snapshot_id);
// 현재 상태의 스냅샷 생성
let certificates = self.store.certificates_up_to_round(round)?;
let snapshot_data = SnapshotData {
round,
certificates,
timestamp: SystemTime::now(),
};
// 스냅샷 저장
self.save_snapshot(&snapshot_path, &snapshot_data).await?;
```

```
// 오래된 스냅샷 정리
self.cleanup_old_snapshots().await?;
Ok(snapshot_id)
}
pub  async  fn  restore_from_snapshot(&self, snapshot_id: &SnapshotId) -> Result
let  snapshot_path = self.get_snapshot_path(snapshot_id);
let  snapshot_data = self.load_snapshot(&snapshot_path).await?;
// 저장소 초기화
self.store.clear()?;
// 스냅샷 데이터 복원
for  certificate  in  snapshot_data.certificates {
self.store.write_certificate(certificate)?;
}
Ok(())
}
}
```

8.7.2 점진적 백업

```
pub  struct IncrementalBackup {
last_backup_round: Round,
backup_storage: BackupStorage,
compression_enabled: bool,
```

```
}

impl IncrementalBackup {
pub  async  fn  perform_incremental_backup(&mut  self, current_round: Round) ->
// 마지막 백업 이후의 변경사항만 백업
let  certificates = self.store.certificates_after_round(self.last_backup_round)?
if !certificates.is_empty() {
let  backup_data = IncrementalBackupData {
start_round: self.last_backup_round + 1,
end_round: current_round,
certificates,
timestamp: SystemTime::now(),
};
// 백업 저장
let  backup_id = self.backup_storage.store_incremental(backup_data).await?;
self.last_backup_round = current_round;
tracing::info!("Incremental backup completed: {}", backup_id);
}
Ok(())
}
}
```

이러한 저장소 시스템을 통해 Narwhal은 데이터의 영속성과 일관성을 보장하면서도 높은 성능을 유지할 수 있습니다.